6

Program Correctness: Strategies

The basis of our approach is the notion of an interpretation of a program: that is, an association of a proposition with each connection in the flow of control through a program.

— Robert W. Floyd
Assigning Meanings to Programs, 1967

As in other applications of mathematical induction (see Example 4.1), the main challenge in applying the inductive assertion method is discovering extra information to make the inductive argument succeed. Consider a typical partial correctness property that asserts that a function's output satisfies some relation with its input. Assuming this property as the inductive hypothesis does not provide any information about how the function behaves between entry and exit. It is a weak hypothesis. Therefore, one must provide more information about the function in the form of additional program annotations. Section 6.1 discusses strategies for discovering this additional information. Section 6.2 applies the strategies to prove that QuickSort always returns a sorted array.

6.1 Developing Inductive Annotations

The machinery presented in Section 5.2 automatically reduces an annotated function to a finite set of verification conditions. Furthermore, the decision procedures discussed in Part II of this text make deciding the validity of many verification conditions an automatable task. For example, all verification conditions in this text fall within decidable fragments of FOL, unless otherwise noted. Thus, determining if a program's annotations are inductive can be automated in many cases. Ongoing research in decision procedures continually expands the set of annotations that produce decidable verification conditions.

Developing annotations is a different matter. As expected, writing a function specification requires human ingenuity, just as implementing the function

requires it. Of course, simple assertions, such as that a program is free of runtime errors, can be generated automatically.

Writing the loop invariants also requires human ingenuity. A certain level of human intervention is acceptable: the programmer ought to know certain facts about her/his code. Loop invariants often capture insights into how the code works and what it accomplishes. Developing the implementation and annotations simultaneously results in more robust systems. Finally, annotations formally document code, facilitating better development in team projects.

However, Section 5.2.5 points out a fundamental limitation of the inductive assertion method of program verification: loop invariants must be inductive for the corresponding verification conditions to be valid, not just invariant. Consequently, the programmer can assert many facts that are indeed invariant; yet if the annotations are not inductive, the facts cannot be proved.

Much research addresses automatic (inductive) invariant discovery. For example, algorithms exist for discovering linear and polynomial relations among integer and real variables. Such invariants can, for example, provide loop index bounds, prove the lack of division by 0, or prove that an index into an array is within bounds. Other methods exist for discovering the "shape" of memory in programming languages with pointers, allowing, for example, the partially automated analysis of linked lists. One of the most important roles of automatic invariant discovery is strengthening the programmer's annotations into inductive annotations. Chapter 12 introduces invariant generation procedures. However, no set of algorithms will ever fully replace humans in writing verified software.

In this section, we suggest structured techniques for developing inductive annotations to prove partial correctness. We emphasize that the methods are just heuristics: human ingenuity is still the most important ingredient in forming proofs.

6.1.1 Basic Facts

To begin a proof, include basic facts in loop invariants. Basic facts include loop index ranges and other "obvious" facts. To be inductive, complex assertions usually require these basic facts. We illustrate the development of basic facts through several examples.

Example 6.1. Consider the loop of LinearSearch(see also Figure 5.1):

```
for
  @L : ⊤
  (int i := ℓ; i ≤ u; i := i + 1) {
  if (a[i] = e) return true;
}
```

Based on the initialization of i, the loop guard, and that i is only modified by being incremented in the loop update, we know that at L,

$$\ell \leq i \leq u + 1 \ .$$

Notice the upper bound. It is a common mistake to forget that on the final iteration, the loop guard is not true. Our basic annotation of the loop is the following:

```
for
  @L : ℓ ≤ i ≤ u + 1
  (int i := ℓ; i ≤ u; i := i + 1) {
  if (a[i] = e) return true;
}
```

■

Example 6.2. Consider the loops of BubbleSort (see also Figure 5.3):

```
for
  @L1 : ⊤
  (int i := |a| − 1; i > 0; i := i − 1) {
  for
    @L2 : ⊤
    (int j := 0; j < i; j := j + 1) {
    if (a[j] > a[j + 1]) {
      int t := a[j];
      a[j] := a[j + 1];
      a[j + 1] := t;
    }
  }
}
```

The outer loop index i ranges according to

$$-1 \leq i < |a|$$

Why -1? If $|a| = 0$, then $|a| - 1 = -1$ so that i is initially -1. Keep in mind that "corner cases" like this one are just as important as normal cases (and perhaps even more important when considering correctness: corner cases are often the source of bugs). In the inner loop, the range of i is more restricted:

$$0 < i < |a|$$

because of the outer loop guard.

In the inner loop, j ranges according to

$$0 \leq j \leq i \ .$$

Therefore, our basic annotation of the two loops is the following:

```
for
  @L1 : −1 ≤ i < |a|
  (int i := |a| − 1; i > 0; i := i − 1) {
  for
    @L2 : 0 < i < |a| ∧ 0 ≤ j ≤ i
    (int j := 0; j < i; j := j + 1) {
    if (a[j] > a[j + 1]) {
      int t := a[j];
      a[j] := a[j + 1];
      a[j + 1] := t;
    }
  }
}
```

Note that the loops modify just the elements of a, not a itself. Therefore, we could add the annotation

$$|a| = |a_0|$$

to both loops. Such an annotation would be useful if the postcondition asserted, for example, that

$$|rv| = |a_0| \ .$$

For the property that we address ($\mathsf{sorted}(rv, 0, |rv| - 1)$), this annotation is not useful. ■

6.1.2 The Precondition Method

Basic facts provide a foundation for more interesting information. The **precondition method** (also called the "backward substitution" or "backward propagation" method) is a strategy for developing more interesting information in a structured way. Again, we emphasize that the method is a heuristic, not an algorithm: it provides some guidance for the human rather than replacing the human's intuition and ingenuity.

The precondition method consists of the following steps:

1. Identify a fact F that is known at one location L in the function ($@L : \ F$) but that is not supported by annotations earlier in the function.
2. Repeat:
   a) Compute the weakest preconditions of F backward through the function, ending at loop invariants or at the beginning of the function.
   b) At each new annotation location L', generalize the new facts to new formula F' ($@L' : \ F'$).

We illustrate the technique through examples.

Example 6.3. Consider the loop of LinearSearch (see also Figure 5.1), annotated with basic facts:

```
for
  @L : ℓ ≤ i ≤ u + 1
  (int i := ℓ; i ≤ u; i := i + 1) {
  if (a[i] = e) return true;
}
return false;
```

The postcondition of LinearSearch is

$$rv \;\leftrightarrow\; \exists i.\; \ell \leq i \leq u \;\wedge\; a[i] = e\;.$$

Consider basic path **(4)** of Example 5.13 but with the current loop invariant substituted for the first assertion:

(4)

$@L:\; F_1:\; \ell \leq i \leq u+1$
$S_1:\; \texttt{assume}\ i > u;$
$S_2:\; rv := \texttt{false};$
$@\mathsf{post}\ F_2:\; rv \;\leftrightarrow\; \exists j.\; \ell \leq j \leq u \;\wedge\; a[j] = e$

Note that we continue to number basic paths as they were numbered in Example 5.13. The VC

$$\{F_1\}S_1; S_2\{F_2\}:\; \ell \leq i \leq u+1 \;\wedge\; i > u \;\rightarrow\; \neg(\exists j.\; \ell \leq j \leq u \;\wedge\; a[j] = e)$$

is not $(T_{\mathbb{Z}} \cup T_{\mathsf{A}})$-valid. Essentially, the antecedent does not assert anything useful about the content of a. Write the consequent as

$$F:\; \forall j.\; \ell \leq j \leq u \;\rightarrow\; a[j] \neq e$$

by pushing in the negation. F says that if LinearSearch exits via S_2, then no element of a in the range $[\ell, u]$ is e. But F is not supported by the current loop invariant at L. In short, F_2 is a fact that is not supported by earlier annotations.

Having identified an unsupported fact, we compute preconditions. To propagate F_2 back to the loop invariant, compute

$$\begin{aligned}
&\mathsf{wp}(F_2,\; S_1; S_2)\\
&\Leftrightarrow\; \mathsf{wp}(\mathsf{wp}(F_2,\; rv := \texttt{false}),\; S_1)\\
&\Leftrightarrow\; \mathsf{wp}(F_2\{rv \mapsto \texttt{false}\},\; S_1)\\
&\Leftrightarrow\; \mathsf{wp}(F_2\{rv \mapsto \texttt{false}\},\; \texttt{assume}\ i > u)\\
&\Leftrightarrow\; i > u \;\rightarrow\; F_2\{rv \mapsto \texttt{false}\}\\
&\Leftrightarrow\; i > u \;\rightarrow\; \forall j.\; \ell \leq j \leq u \;\rightarrow\; a[j] \neq e
\end{aligned}$$

The final formula

$$G:\ i > u\ \rightarrow\ \forall j.\ \ell \le j \le u\ \rightarrow\ a[j] \neq e\ ,$$

in particular the antecedent $i > u$, and some intuition suggests the generalization

$$G':\ \forall j.\ \ell \le j < i\ \rightarrow\ a[j] \neq e\ .$$

We compute one backward iteration through the loop to increase our confidence:

(3)

$@L:\ H:?$
$S_1:\ \texttt{assume}\ i \le u;$
$S_2:\ \texttt{assume}\ a[i] \neq e;$
$S_3:\ i := i+1;$
$@L:\ G:\ i > u\ \rightarrow\ \forall j.\ \ell \le j \le u\ \rightarrow\ a[j] \neq e$

Then

$$\begin{aligned}
&\mathsf{wp}(G,\ S_1; S_2; S_3)\\
&\Leftrightarrow\ \mathsf{wp}(\mathsf{wp}(G,\ i := i+1),\ S_1; S_2)\\
&\Leftrightarrow\ \mathsf{wp}(G\{i := i+1\},\ S_1; S_2)\\
&\Leftrightarrow\ \mathsf{wp}(\mathsf{wp}(G\{i := i+1\},\ \texttt{assume}\ a[i] \neq e),\ S_1)\\
&\Leftrightarrow\ \mathsf{wp}(a[i] \neq e\ \rightarrow\ G\{i := i+1\},\ S_1)\\
&\Leftrightarrow\ \mathsf{wp}(a[i] \neq e\ \rightarrow\ G\{i := i+1\},\ \texttt{assume}\ i \le u)\\
&\Leftrightarrow\ i \le u\ \rightarrow\ a[i] \neq e\ \rightarrow\ G\{i := i+1\}\\
&\Leftrightarrow\ i \le u\ \wedge\ a[i] \neq e\ \wedge\ i+1 > u\ \rightarrow\ \forall j.\ \ell \le j \le u\ \rightarrow\ a[j] \neq e\\
&\Leftrightarrow\ i = u\ \wedge\ a[u] \neq e\ \rightarrow\ \forall j.\ \ell \le j \le u\ \rightarrow\ a[j] \neq e\\
&\Leftrightarrow\ i = u\ \wedge\ a[u] \neq e\ \rightarrow\ \forall j.\ \ell \le j \le u-1\ \rightarrow\ a[j] \neq e\\
&\Leftrightarrow\ i = u\ \wedge\ a[u] \neq e\ \rightarrow\ \forall j.\ \ell \le j \le i-1\ \rightarrow\ a[j] \neq e
\end{aligned}$$

To obtain the second-to-last line from the third-to-last, note that the antecedent already asserts that $a[u] \neq e$; hence, its occurrence as the case $j = u$ of $\forall j.\ \ell \le j \le u \cdots$ is redundant. The final line is realized by applying the equality $i = u$ to the upper bound on j. As we suspected, it seems that the right bound on j should be related to the progress of i, rather than being fixed to u. This observation from computing the weakest precondition matches our intuition. One trick to generalize assertions is to *replace fixed terms (bounds, indices, etc.) with terms that evolve according to the loop counter.*

Thus, we settle on the formula

$$G':\ \forall j.\ \ell \le j < i\ \rightarrow\ a[j] \neq e\ .$$

That is, all previously checked entries of a do not equal e. We add this assertion to the loop invariant:

```
for
  @L: ℓ ≤ i ≤ u+1 ∧ (∀j. ℓ ≤ j < i → a[j] ≠ e)
  (int i := ℓ; i ≤ u; i := i+1) {
  if (a[i] = e) return true;
}
```

The result is similar to the annotation in Figure 5.15. Generating and checking the corresponding VCs reveals that the annotations are inductive. ■

Example 6.4. Consider the version of BinarySearch of Figure 5.12 that contains runtime assertions but has only a trivial function specification $\top$. Using the precondition method, we infer a function precondition that makes the annotations inductive. Contexts that call BinarySearch are then forced to obey this function precondition, guaranteeing a lack of runtime errors.

Consider the path from function entry to the assertion protecting the array access:

$(\cdot)$

$@\mathsf{pre}\ H : ?$
$S_1 :\ \texttt{assume}\ \ell \leq u;$
$S_2 :\ m := (\ell + u)\ \texttt{div}\ 2;$
$@\ F :\ 0 \leq m < |a|$

Compute

$$\begin{aligned}
&\mathsf{wp}(F,\ S_1; S_2)\\
&\Leftrightarrow\ \mathsf{wp}(\mathsf{wp}(F,\ m := (\ell + u)\ \texttt{div}\ 2),\ S_1)\\
&\Leftrightarrow\ \mathsf{wp}(F\{m \mapsto (\ell + u)\ \texttt{div}\ 2\},\ S_1)\\
&\Leftrightarrow\ \mathsf{wp}(F\{m \mapsto (\ell + u)\ \texttt{div}\ 2\},\ \texttt{assume}\ \ell \leq u)\\
&\Leftrightarrow\ \ell \leq u\ \rightarrow\ F\{m \mapsto (\ell + u)\ \texttt{div}\ 2\}\\
&\Leftrightarrow\ \ell \leq u\ \rightarrow\ 0 \leq (\ell + u)\ \texttt{div}\ 2 < |a|\\
&\Leftarrow\ 0 \leq \ell\ \wedge\ u < |a|
\end{aligned}$$

The final line implies the penultimate line, for if $0 \leq \ell\ \wedge\ u < |a|$ and $\ell \leq u$, then both $0 \leq \ell < |a|$ and $0 \leq u < |a|$; hence, their mean is also in the range $[0, |a| - 1]$. Therefore, it is guaranteed that

$$0 \leq \ell\ \wedge\ u < |a|\ \rightarrow\ \mathsf{wp}(F,\ S_1; S_2)$$

is $T_{\mathbb{Z}}$-valid.

The formula $0 \leq \ell\ \wedge\ u < |a|$ appears as the function precondition in Figure 6.1. The annotations are inductive, proving that the runtime assertion $0 \leq m < |a|$ holds in every execution of BinarySearch in which the precondition $0 \leq \ell\ \wedge\ u < |a|$ is satisfied. ■

Example 6.5. Consider the following code fragment of BubbleSort (see also Figure 5.3)

```
@pre 0 ≤ ℓ ∧ u < |a|
@post ⊤
bool BinarySearch(int[] a, int ℓ, int u, int e) {
  if (ℓ > u) return false;
  else {
    @ 2 ≠ 0;
    int m := (ℓ + u) div 2;
    @ 0 ≤ m < |a|;
    if (a[m] = e) return true;
    else if (a[m] < e) return BinarySearch(a, m + 1, u, e);
    else return BinarySearch(a, ℓ, m − 1, e);
  }
}
```

Fig. 6.1. BinarySearch with runtime assertions

```
for
  @L1 : −1 ≤ i < |a|
  (int i := |a| − 1; i > 0; i := i − 1) {
  for
    @L2 : 0 < i < |a| ∧ 0 ≤ j ≤ i
    (int j := 0; j < i; j := j + 1) {
    if (a[j] > a[j + 1]) {
      int t := a[j];
      a[j] := a[j + 1];
      a[j + 1] := t;
    }
  }
}
return a;
```

and its postcondition

$$F : \mathsf{sorted}(rv, 0, |rv| - 1) \ .$$

Consider the path

(6)

```
@L1 : G : ?
S1 : assume i ≤ 0;
S2 : rv := a;
@post F : sorted(rv, 0, |rv| − 1)
```

Computing $\mathsf{wp}(F,\ S_1; S_2)$ produces the formula

$$F' : \ i \leq 0 \ \rightarrow \ \mathsf{sorted}(a, 0, |a| - 1) \ ,$$

which tells us (not surprisingly) that a should be sorted upon exiting the outer loop. Observe the index variable of the outer loop: it starts at $|a| - 1$

and decrements down to 0. Therefore, recalling the trick to *replace fixed terms (bounds, indices, etc.) with terms that evolve according to the loop counter* suggests the following generalization of F':

$$G: \ \mathsf{sorted}(a, i, |a| - 1) \ .$$

G trivially holds upon entering the outer loop; moreover, it follows from the behavior of i that progress is made by working down the array. The outer loop invariant L_1 should include G. Thus, we have

$$@L_1: \ -1 \leq i < |a| \ \wedge \ \mathsf{sorted}(a, i, |a| - 1)$$

so far.

Propagate G via wp to the inner loop along the path from the exit of the inner loop L_2 to the top of the outer loop L_1:

(5)

$@L_2: \ H: ?$
$S_1: \ \texttt{assume}\ j \geq i;$
$S_2: \ i := i - 1;$
$@L_1: \ G: \ \mathsf{sorted}(a, i, |a| - 1)$

The result at L_2 is the formula

$$H': \ j \geq i \ \rightarrow \ \mathsf{sorted}(a, i - 1, |a| - 1) \ ,$$

which states that when the inner loop has *finished*, the range $[i - 1, |a| - 1]$ is sorted. Immediately generalizing H' to

$$H'': \ \mathsf{sorted}(a, i - 1, |a| - 1)$$

is too strong. For suppose H'' were to annotate the inner loop at L_2, and consider the path

(2)

$@L_1: \ G: \ \mathsf{sorted}(a, i, |a| - 1)$
$S_1: \ \texttt{assume}\ i > 0;$
$S_2: \ j := 0;$
$@L_2: \ H'': \ \mathsf{sorted}(a, i - 1, |a| - 1)$

Computing

$$G \ \rightarrow \ \mathsf{wp}(H'', \ \texttt{assume}\ i > 0; \ j := 0)$$

produces

$$\mathsf{sorted}(a, i, |a| - 1) \ \wedge \ i > 0 \ \rightarrow \ \mathsf{sorted}(a, i - 1, |a| - 1) \ ,$$

which is not $(T_{\mathbb{Z}} \cup T_{\mathsf{A}})$-valid. All we know at L_1 (with respect to sortedness of a) is $G: \ \mathsf{sorted}(a, i, |a| - 1)$. Essentially, $\mathsf{sorted}(a, i - 1, |a| - 1)$ at L_2 is a special

case that definitely holds only when the inner loop has finished. Therefore, we generalize H' to the weaker assertion H : $\mathsf{sorted}(a, i, |a| - 1)$, which claims that a smaller subrange of a is sorted.

At this point, we have annotated the loops of BubbleSort as follows:

```
for
  @L1 : −1 ≤ i < |a| ∧ sorted(a, i, |a| − 1)
  (int i := |a| − 1; i > 0; i := i − 1) {
  for
    @L2 : 0 < i < |a| ∧ 0 ≤ j ≤ i ∧ sorted(a, i, |a| − 1)
    (int j := 0; j < i; j := j + 1) {
    if (a[j] > a[j + 1]) {
      int t := a[j];
      a[j] := a[j + 1];
      a[j + 1] := t;
    }
  }
}
```

The resulting VCs are not valid. Further annotations require some insight on our part, which leads us to the next section. ■

6.1.3 A Strategy

In general, proofs require insights beyond generalizing formulae obtained through the precondition method. We adopt the following strategy when proving partial correctness.

First, decompose the function specification into atomic properties. Then analyze each atomic property. For example, to prove that BubbleSort returns a sorted array that is a permutation of the input, study the sortedness property and permutation property separately. In some cases, several atomic properties may have to be examined together to complete the proof. For each basic property, apply the following steps:

1. Assert basic facts (Section 6.1.1).
2. Repeat:
   a) Use the precondition method to propagate annotations (Section 6.1.2).
   b) Formalize an insight.

The second step suggests applying the precondition method until nothing more can be learned. Then pause, understand another essential fact about the program, and resume applying the precondition method.

While Chapter 12 discusses the foundations of algorithms for automatically generating inductive annotations, the reader should be aware that even the best of these algorithms cannot approach the abilities of a human. Take heart! Experience has shown that students quickly become adept at annotating programs.

Example 6.6. We resume our analysis of BubbleSort from Example 6.5. Some cogitation (and observation of sample traces; see Figure 5.4) suggests that BubbleSort exhibits the following behavior: the inner loop propagates the largest value of the unsorted region to the right side of the unsorted region, thus expanding the sorted region. At every iteration, j is the index of the largest value found so far. In other words, all values in the range $[0, j-1]$ are at most $a[j]$:

$$F: \ \mathsf{partitioned}(a, 0, j-1, j, j) \ .$$

This observations should be added as an annotation at L_2. Having gained new insight into BubbleSort, we return to the precondition method and propagate F back to the outer loop at L_1 along the path

(2)

$@L_1: \ H: ?$
$S_1: \ \texttt{assume}\ i > 0;$
$S_2: \ j := 0;$
$@L_2: \ F: \ \mathsf{partitioned}(a, 0, j-1, j, j)$

resulting in the new annotation

$$\mathsf{wp}(F, \ S_1; S_2): \ i > 0 \ \rightarrow \ \mathsf{partitioned}(a, 0, -1, 0, 0)$$

at L_1. The result is trivially valid according to the definition of partitioned, so it does not contribute any new information. Thus, we finish this round of Step 2 with the annotations

```
for
  @L1 : −1 ≤ i < |a| ∧ sorted(a, i, |a| − 1)
  (int i := |a| − 1; i > 0; i := i − 1) {
  for
    @L2 : [0 < i < |a| ∧ 0 ≤ j ≤ i
           ∧ partitioned(a, 0, j − 1, j, j) ∧ sorted(a, i, |a| − 1)]
    (int j := 0; j < i; j := j + 1) {
    if (a[j] > a[j + 1]) {
      int t := a[j];
      a[j] := a[j + 1];
      a[j + 1] := t;
    }
  }
}
```

The annotations are not yet inductive.

Some further meditation enlightens us with the following: the sorted region must contain the largest elements of a, for the inner loop has assiduously moved the largest element of the unsorted region to the sorted region. In other words, the sorted range $[i+1, |a|-1]$ contains the largest elements of a:

$$G: \ \mathsf{partitioned}(a, 0, i, i+1, |a|-1) \ .$$

This observation should be added as an annotation at L_1. Now we propagate G from L_1 to L_2. Recall from Example 6.5 that our propagation of $\mathsf{sorted}(a, i, |a|-1)$ to the inner loop was unsuccessful. Similarly,

$$\mathsf{wp}(G, \ \texttt{assume}\ j \geq i;\ i := i-1) \\ \Leftrightarrow \ j \geq i \ \rightarrow \ \mathsf{partitioned}(a, 0, i-1, i, |a|-1)$$

for the path

(5)

$@L_2: \ H: ?$
$\texttt{assume}\ j \geq i;$
$i := i-1;$
$@L_1: \ G: \ \mathsf{partitioned}(a, 0, i, i+1, |a|-1)$

cannot be generalized to

$$\mathsf{partitioned}(a, 0, i-1, i, |a|-1) \ .$$

Instead, consider the path from L_1 to L_2:

(2)

$@L_1: \ G: \ \mathsf{partitioned}(a, 0, i, i+1, |a|-1)$
$S_1: \ \texttt{assume}\ i > 0;$
$S_2: \ j := 0;$
$@L_2: \ H: ?$

Find the strongest formula H that can annotate the inner loop such that the VC

$$\mathsf{partitioned}(a, 0, i, i+1, |a|-1) \ \rightarrow \ \mathsf{wp}(H, \ S_1; S_2)$$

for the path is valid. In other words, seek a formula H annotating the inner loop that is supported by the annotation G of the outer loop. The strongest such formula is G itself.

These new annotations result in Figure 5.17. ■

6.2 Extended Example: QuickSort

In this section, we bring together the concepts studied in this and the last chapters through a single example. We prove that QuickSort always halts and returns a sorted array. We argue at the level of annotations, leaving a computer or the reader to check the VCs.

Figure 6.2 lists the high-level functions of QuickSort. QuickSort is a wrapper function (or public interface) for the recursive function qsort, which sorts array a_0 in the range $[\ell, u]$. As in BubbleSort, the first line of qsort assigns a_0 to an

```
typedef struct qs {
  int pivot;
  int[] array;
} qs;

@pre ⊤
@post sorted(rv, 0, |rv| − 1)
int[] QuickSort(int[] a) {
  return qsort(a, 0, |a| − 1);
}

@pre ⊤
@post ⊤
int[] qsort(int[] a0, int ℓ, int u) {
  int[] a := a0;
  if (ℓ ≥ u) return a;
  else {
    qs p := partition(a, ℓ, u);
    a := p.array;
    a := qsort(a, ℓ, p.pivot − 1);
    a := qsort(a, p.pivot + 1, u);
    return a;
  }
}
```

Fig. 6.2. Main functions of QuickSort

array a because qsort modifies a (recall that pi does not allow parameters to be modified). The qs data structure holds the two data that the partition function, listed in Figure 6.3, returns: the pivot index *pivot* and the partitioned array *array*.

One level of recursion of qsort works as follows. If $\ell \geq u$, then the trivial range $[\ell, u]$ of a_0 is already sorted. Otherwise, partition chooses a *pivot index* $pi \in [\ell, u]$, remembering the *pivot value* $a[pi]$ as pv. It then swaps cells pi and u of a so that the randomly chosen pivot now appears on the right side of the $[\ell, u]$ subarray. random has the following prototype:

```
@pre ℓ ≤ u
@post ℓ ≤ rv ≤ u
int random(int ℓ, int u);
```

The for loop of partition partitions a such that all elements at most pv are on the left and all elements greater than pv are on the right. Within the loop, $j < u$, so that the pivot value pv, stored in $a[u]$, remains untouched. When the loop finishes, if $i < u - 1$, then the value $a[i + 1]$ is the first value greater than pv; otherwise, all elements of a are at most pv. Finally, partition

```
@pre ⊤
@post ⊤
qs partition(int[] a0, int ℓ, int u) {
  int[] a := a0;
  int pi := random(ℓ, u);
  int pv := a[pi];
  a[pi] := a[u];
  a[u] := pv;

  int i := ℓ − 1;
  for @ ⊤
    (int j := ℓ; j < u; j := j + 1) {
    if (a[j] ≤ pv) {
      i := i + 1;
      t := a[i];
      a[i] := a[j];
      a[j] := t;
    }
  }

  t := a[i + 1];
  a[i + 1] := a[u];
  a[u] := t;
  return
    { pivot = i + 1;
      a = a;
    };
}
```

Fig. 6.3. QuickSort's partition function

swaps the pivot value $a[u]$ with $a[i+1]$ so that a is partitioned as follows in the range $[\ell, u]$: cells to the left of $i+1$ have value at most pv; $a[i+1] = pv$; and cells to the right of $i+1$ have value greater than pv. It returns the pivot index $i+1$ and the partitioned array a via an instance of the qs data type.

Finally, qsort recursively sorts the subarrays to the left and to the right of the pivot index.

Figure 6.4 presents a sample trace. In the first line, partition chooses the second cell as the pivot and swaps it with cell u. The subsequent six lines follow the partition's loop as it partitions elements according to pv. The penultimate line shows the swap that brings the pivot element into the pivot position. The final line shows the state of the array when it is returned to qsort. qsort calls itself recursively on the two indicated subarrays. We encourage the reader to understand QuickSort and the sample trace before reading further.

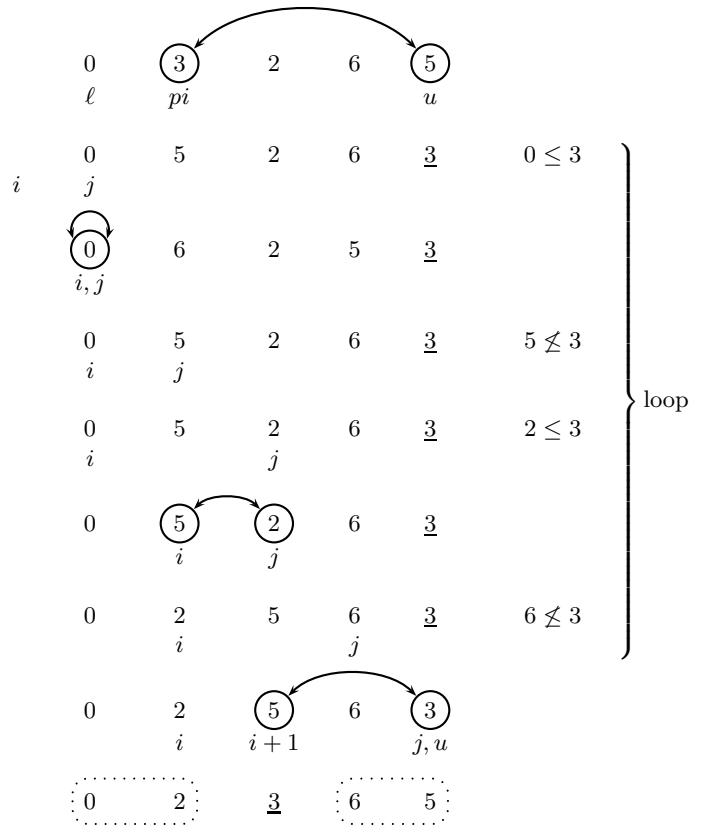

Fig. 6.4. Sample execution of QuickSort

6.2.1 Partial Correctness

We prove that if QuickSort halts, then it returns a sorted array. First, we develop the function specifications for qsort and partition so that QuickSort and qsort have inductive annotations. We then leave the annotation of the loop of partition as Exercise 6.3.

First, annotate qsort and partition with their function specifications. To avoid runtime errors, the function preconditions should include

$$0 \leq \ell \ \wedge \ u < |a_0| \ .$$

Next, while the returned array is not the same as the input array a_0 in either function, we know that their lengths are the same:

$$|rv| = |a_0| \ .$$

We also observe that neither qsort nor partition modify the array outside of the range $[\ell, u]$. Thus, we note in the function postcondition that

$$\mathsf{beq}(rv, a_0, 0, \ell - 1) \ \wedge \ \mathsf{beq}(rv, a_0, u + 1, |a_0| - 1) \ .$$

The **bounded equality** predicate beq is defined

$$\mathsf{beq}(a, b, k_1, k_2) \ \Leftrightarrow \ \forall i. \ k_1 \leq i \leq k_2 \ \rightarrow \ a[i] = b[i]$$

in the theory $T_{\mathbb{Z}} \cup T_{\mathsf{A}}$. It asserts that two arrays are equal in the index range $[k_1, k_2]$.

The annotations for partition vary slightly because of its return type:

$$|rv.array| = |a_0| \ \wedge \ \mathsf{beq}(rv.array, a_0, 0, \ell - 1) \\ \wedge \ \mathsf{beq}(rv.array, a_0, u + 1, |a_0| - 1)$$

Since partition returns an integer ($pivot$) and an array ($array$) in a `qs` structure, the postcondition asserts facts about $rv.array$.

To finish with qsort, formalize that qsort sorts the range $[\ell, u]$ of the given array:

$$\mathsf{sorted}(rv, \ell, u) \ .$$

So far, then, we have specified qsort as follows:

$$@\mathsf{pre} \ 0 \leq \ell \ \wedge \ u < |a_0|$$
$$@\mathsf{post} \left[\begin{array}{l} |rv| = |a_0| \ \wedge \ \mathsf{beq}(rv, a_0, 0, \ell - 1) \ \wedge \ \mathsf{beq}(rv, a_0, u + 1, |a_0| - 1) \\ \wedge \ \mathsf{sorted}(rv, \ell, u) \end{array} \right]$$
$$\texttt{int[]} \ \mathsf{qsort}(\texttt{int[]} \ a_0, \ \texttt{int} \ \ell, \ \texttt{int} \ u)$$

To finish the specification of partition, recall that partition is intended to return an array that is partitioned around the pivot element. Therefore, let us formalize the description that we gave above. Observe that the specification of random guarantees that

$$\ell \leq rv.pivot \leq u \ ,$$

as desired. Now, for the left subarray,

$$\forall i. \ \ell \leq i < rv.pivot \ \rightarrow \ rv.array[i] \leq rv.array[rv.pivot] \ ,$$

or, as a partition,

$$\mathsf{partitioned}(rv.array, \ell, rv.pivot - 1, rv.pivot, rv.pivot) \ ,$$

while for the right subarray,

$$\forall i. \ rv.pivot < i \leq u \ \rightarrow \ rv.array[rv.pivot] < rv.array[i] \ .$$

We weaken this assertion slightly to

$$\mathsf{partitioned}(rv.array, rv.pivot, rv.pivot, rv.pivot + 1, u) \ ,$$

which does not capture the strict inequality but is more convenient for reasoning. For partition, we have thus specified the following:

$$
\begin{array}{l}
@\mathsf{pre}\ 0 \le \ell\ \wedge\ u < |a_0| \\
@\mathsf{post}\begin{bmatrix} |rv.array| = |a_0|\ \wedge\ \mathsf{beq}(rv.array, a_0, 0, \ell - 1) \\ \quad \wedge\ \mathsf{beq}(rv.array, a_0, u+1, |a_0| - 1) \\ \quad \wedge\ \ell \le rv.pivot \le u \\ \quad \wedge\ \mathsf{partitioned}(rv.array, \ell, rv.pivot - 1, rv.pivot, rv.pivot) \\ \quad \wedge\ \mathsf{partitioned}(rv.array, rv.pivot, rv.pivot, rv.pivot + 1, u) \end{bmatrix} \\
\mathtt{qs}\ \mathsf{partition}(\mathtt{int[]}\ a_0,\ \mathtt{int}\ \ell,\ \mathtt{int}\ u)
\end{array}
$$

Let us step back a moment. Essentially, we have specified that qsort does not modify the array outside of the range $[\ell, u]$. Regarding the subarray given by $[\ell, u]$, all we have asserted is that it is sorted in the returned array. Focus on the recursive calls to qsort: is knowing that the ranges $[\ell, p.pivot - 1]$ and $[p.pivot + 1, u]$ of a are sorted enough to conclude that the range $[\ell, u]$ is sorted when a is returned? In other words, is the VC corresponding to the following basic path valid? The basic path follows the path from the precondition to the second `return` statement, using the function call abstraction introduced in Section 5.2.1 to abstract away functions calls:

$$
\begin{array}{l}
(\cdot) \\
@\mathsf{pre}\ 0 \le \ell\ \wedge\ u < |a_0| \\
a := a_0; \\
\mathtt{assume}\begin{bmatrix} |v_1.array| = |a|\ \wedge\ \mathsf{beq}(v_1.array, a, 0, \ell - 1) \\ \quad \wedge\ \mathsf{beq}(v_1.array, a, u+1, |a| - 1) \\ \quad \wedge\ \ell \le v_1.pivot \le u \\ \quad \wedge\ \mathsf{partitioned}(v_1.array, \ell, v_1.pivot - 1, v_1.pivot, v_1.pivot) \\ \quad \wedge\ \mathsf{partitioned}(v_1.array, v_1.pivot, v_1.pivot, v_1.pivot + 1, u) \end{bmatrix}; \\
p := v_1; \\
a := p.array; \\
\mathtt{assume}\begin{bmatrix} |v_2| = |a|\ \wedge\ \mathsf{beq}(v_2, a, 0, \ell - 1)\ \wedge\ \mathsf{beq}(v_2, a, p.pivot, |a| - 1) \\ \quad \wedge\ \mathsf{sorted}(v_2, \ell, p.pivot - 1) \end{bmatrix}; \\
a := v_2; \\
\mathtt{assume}\begin{bmatrix} |v_3| = |a|\ \wedge\ \mathsf{beq}(v_3, a, 0, p.pivot)\ \wedge\ \mathsf{beq}(v_3, a, u+1, |a| - 1) \\ \quad \wedge\ \mathsf{sorted}(v_3, p.pivot + 1, u) \end{bmatrix}; \\
a := v_3; \\
rv := a; \\
@\begin{bmatrix} |rv| = |a_0|\ \wedge\ \mathsf{beq}(rv, a_0, 0, \ell - 1)\ \wedge\ \mathsf{beq}(rv, a_0, u+1, |a_0| - 1) \\ \quad \wedge\ \mathsf{sorted}(rv, \ell, u) \end{bmatrix}
\end{array}
$$

The corresponding VC is not valid. The assumptions about v_1, v_2, and v_3 are not strong enough to imply that the range $[\ell, u]$ of rv is sorted.

The standard approach to addressing this problem is to reason simultaneously that qsort returns an array that is a permutation of its input array

(a permuted array contains the same elements as the original array but possibly in a different order). However, reasoning about permutations presents a problem. A straightforward formalization of permutation is not possible in FOL, instead requiring second-order logic. We could assert that the output is a **weak permutation** of the input: all values occurring in the input array occur in the output array but possibly with a varying number of occurrences. Formally,

$$\forall e.\ (\exists i.\ a_0[i] = e) \ \leftrightarrow\ (\exists j.\ rv[j] = e)\ .$$

In Exercise 6.5, we ask the reader to explore an approximation to weak permutation.

However, that the elements are permuted is a stronger statement than necessary to prove sortedness. Instead, notice that QuickSort imposes a larger partitioning of the intermediate arrays than we have previously observed in our analysis. At every level of recursion of qsort, the elements of a_0 in the range $[\ell, u]$ are at least the elements to their left and at most the elements to their right. Formally, we strengthen the specification of qsort as follows:

$$\begin{array}{l}
\text{@pre} \left[\begin{array}{l} 0 \le \ell \ \wedge\ u < |a_0| \\ \ \wedge\ \mathsf{partitioned}(a_0, 0, \ell - 1, \ell, u) \\ \ \wedge\ \mathsf{partitioned}(a_0, \ell, u, u + 1, |a_0| - 1) \end{array}\right] \\
\text{@post} \left[\begin{array}{l} |rv| = |a_0| \ \wedge\ \mathsf{beq}(rv, a_0, 0, \ell - 1) \ \wedge\ \mathsf{beq}(rv, a_0, u + 1, |a_0| - 1) \\ \ \wedge\ \mathsf{partitioned}(rv, 0, \ell - 1, \ell, u) \\ \ \wedge\ \mathsf{partitioned}(rv, \ell, u, u + 1, |rv| - 1) \\ \ \wedge\ \mathsf{sorted}(rv, \ell, u) \end{array}\right] \\
\texttt{int[]}\ \mathsf{qsort}(\texttt{int[]}\ a_0,\ \texttt{int}\ \ell,\ \texttt{int}\ u)
\end{array}$$

Of course, now the specification of partition must be strengthened to carry this reasoning through the main basic path of qsort:

$$\begin{array}{l}
\text{@pre} \left[\begin{array}{l} 0 \le \ell \ \wedge\ u < |a_0| \\ \ \wedge\ \mathsf{partitioned}(a_0, 0, \ell - 1, \ell, u) \\ \ \wedge\ \mathsf{partitioned}(a_0, \ell, u, u + 1, |a_0| - 1) \end{array}\right] \\
\text{@post} \left[\begin{array}{l} |rv.array| = |a_0| \ \wedge\ \mathsf{beq}(rv.array, a_0, 0, \ell - 1) \\ \ \wedge\ \mathsf{beq}(rv.array, a_0, u + 1, |a_0| - 1) \\ \ \wedge\ \mathsf{partitioned}(rv.array, 0, \ell - 1, \ell, u) \\ \ \wedge\ \mathsf{partitioned}(rv.array, \ell, u, u + 1, |rv.array| - 1) \\ \ \wedge\ \ell \le rv.pivot \le u \\ \ \wedge\ \mathsf{partitioned}(rv.array, \ell, rv.pivot - 1, rv.pivot, rv.pivot) \\ \ \wedge\ \mathsf{partitioned}(rv.array, rv.pivot, rv.pivot, rv.pivot + 1, u) \end{array}\right] \\
\mathsf{qs\ partition}(\texttt{int[]}\ a_0,\ \texttt{int}\ \ell,\ \texttt{int}\ u)
\end{array}$$

That is, partition preserves this partitioning even as it manipulates the elements in the range $[\ell, u]$. Indeed, partition itself imposes the necessary parti-

tioning for the next level of recursion, which we already observed earlier as the final partitioned assertions of the function postcondition.

The annotations of QuickSort and qsort are inductive. Exercise 6.3 asks the reader to finish the proof by annotating the `for` loop of partition so that the annotations of partition are also inductive.

6.2.2 Total Correctness

To prove total correctness — that QuickSort actually returns a sorted arrays — we need to prove that QuickSort always halts. Our implementation of QuickSort has both recursive behavior in function qsort and looping behavior in function partition. We must analyze both possible sources of nontermination.

To prove that the loop in partition always halts, let us use the obvious ranking function of $\delta_1 : u - j$ suggested by the structure of the loop. δ_1 clearly maps the program state to $\mathbb{Z}$, but we prove the stronger fact that δ_1 actually maps the program state to $\mathbb{N}$ with well-founded relation $<$. In particular, we prove that

- $u - j \geq 0$ is a loop invariant;
- and $u - j$ decreases on each iteration.

Annotating the loop with the bounds on j suggested by the loop structure proves that the loop always halts:

`for`
$@L_1 : \ \ell \leq j \ \wedge \ j \leq u$
$\downarrow \ \delta_1 : \ u - j$
(`int` $j := \ell;\ j < u;\ j := j + 1$)

Proving that the recursion of qsort always halts is superficially more difficult. The argument that we would like to make is that $u - \ell$ decreases on each recursive call, which requires proving that the pivot value returned by partition lies within the range $[\ell, u]$.

Observe, however, that $u - \ell$ may be negative when qsort is called with $\ell > u$. But in this case, $\ell = u + 1$, for either $|a_0| = 0$, and qsort was called from QuickSort; or $p.pivot = \ell$ or $p.pivot = u$, and qsort was called recursively. More generally, we can establish that $u - \ell + 1 \geq 0$ is an invariant of qsort. Hence, $\delta_2 : u - \ell + 1$ is our proposed ranking function that maps the program states to $\mathbb{N}$ with well-founded relation $<$.

Figure 6.5 formalizes the arguments that δ_1 and δ_2 are ranking functions. Notice that bounds on i are proved as loop invariants at L_1. These bounds imply that $rv.pivot$ lies within the range $[\ell, u]$ as required.

One trick that would avoid reasoning about the case in which $\ell > u$ is to cut the recursion at a point within qsort rather than at function entry. Figure 6.6 provides an alternate argument in which the ranking function labels the

```
@pre u − ℓ + 1 ≥ 0
@post ⊤
↓ δ2 : u − ℓ + 1
int[] qsort(int[] a0, int ℓ, int u) {
  int[] a := a0;
  if (ℓ ≥ u) return a;
  else {
    qs p := partition(a, ℓ, u);
    a := p.array;
    a := qsort(a, ℓ, p.pivot − 1);
    a := qsort(a, p.pivot + 1, u);
    return a;
  }
}

@pre ℓ ≤ u
@post ℓ ≤ rv.pivot ∧ rv.pivot ≤ u
qs partition(int[] a0, int ℓ, int u) {
  ⋮
  int i := ℓ − 1;
  for
    @L1 : ℓ ≤ j ∧ j ≤ u ∧ ℓ − 1 ≤ i ∧ i < j
    ↓ δ1 : u − j
    (int j := ℓ; j < u; j := j + 1) {
    ⋮
  }
  ⋮
  return
    { pivot = i + 1;
      a = a;
    };
}
```

Fig. 6.5. QuickSort always halts

`else` branch in qsort. The first branch terminates the recursion. partition is annotated as in Figure 6.5.

6.3 Summary

This chapter presents strategies for specifying and proving the correctness of sequential programs. It covers:

- *Strategies* for proving partial correctness. The need for strengthening annotations. Basic facts; the precondition method.

```
@pre ⊤
@post ⊤
int[] qsort(int[] a_0, int ℓ, int u) {
  int[] a := a_0;
  if (ℓ ≥ u) return a;
  else {
    ↓ δ_3 : u − ℓ
    qs p := partition(a, ℓ, u);
    a := p.array;
    a := qsort(a, ℓ, p.pivot − 1);
    a := qsort(a, p.pivot + 1, u);
    return a;
  }
}
```

Fig. 6.6. Alternate argument that QuickSort always halts

```
@pre ⊤
@post ∀i. 0 ≤ i < |rv| → rv[i] ≥ 0
int[] abs(int[] a_0) {
  int[] a := a_0;
  for @ ⊤
    (int i := 0; i < |a|; i := i + 1) {
    if (a[i] < 0) {
      a[i] := − a[i];
    }
  }
  return a;
}
```

Fig. 6.7. Computing the absolute value of elements of a_0

- A full proof that QuickSort returns a sorted array.

Bibliographic Remarks

QuickSort was discovered by Tony Hoare, who also proposed specifying and verifying programs using FOL [39].

Exercises

6.1 (Absolute value). Prove the partial correctness of abs in Figure 6.7. That is, annotate the function; list basic paths and verification conditions; and argue that the VCs are valid.

```
@pre ⊤
@post sorted(rv, 0, |rv| − 1)
int[] InsertionSort(int[] a0) {
  int[] a := a0;
  for @ ⊤
    (int i := 1; i < |a|; i := i + 1) {
    int t := a[i];
    for @ ⊤
      (int j := i − 1; j ≥ 0; j := j − 1) {
      if (a[j] ≤ t) break;
      a[j + 1] := a[j];
    }
    a[j + 1] := t;
  }
  return a;
}
```

Fig. 6.8. InsertionSort

6.2 (InsertionSort). Prove the partial correctness of InsertionSort. That is, annotate the function; list basic paths and verification conditions; and argue that the VCs are valid. See Figure 6.8.

As in other languages, the `break` statement moves control to the loop exit.

6.3 (QuickSort). Finish the proof of the sortedness property of QuickSort. That is, annotate partition of Figure 6.3 so that its precondition and postcondition annotations given at the end of Section 6.2 are inductive.

6.4 (MergeSort). Prove the partial correctness of MergeSort. See Figure 6.9. The function merge uses the pi keyword `new`, which allocates an array of the specified size. Therefore, it is known after the allocation to buf that $|buf| = u - \ell + 1$.

First, deduce the function specifications for ms and merge by focusing on MergeSort and ms. Prove MergeSort and ms correct with respect to these annotations. Then analyze merge.

Since MergeSort is fairly long, you need not list basic paths and VCs. Just present MergeSort with its inductive annotations.

6.5 (Weak permutation). Define **weak permutation** as follows:

$$\forall e.\ (\exists i.\ a[i] = e) \ \leftrightarrow\ (\exists j.\ b[j] = e)\ . \tag{6.1}$$

Unfortunately, the decision procedures for arrays discussed in Chapter 11 cannot decide the validity of VCs arising from wperm annotations, as such VCs fall outside of the studied fragments of T_{A}. Instead, we describe an approximation.

Consider annotating BubbleSort as in Figure 6.10. The `define` keyword defines a global constant. In this case, e is defined to have some nondeterministic value; that is, e stands for an arbitrary integer. The annotations then use this e in wperm literals, where wperm is defined as follows:

$$\mathsf{wperm}(a, a_0, e) \;\Leftrightarrow\; (\exists i.\ a[i] = e) \;\leftrightarrow\; (\exists j.\ a_0[j] = e)\ .$$

Compared to the full definition (6.1) of weak permutation, wperm does not have a universally quantified variable e; instead, it uses a given expression e, in this case the global constant e.

(a) Argue that the annotations of Figure 6.10 are inductive. That is, list the VCs and argue their validity.

(b) Argue that the annotations imply that BubbleSort actually satisfies the weakest permutation property. That is, prove that the validity of the VC

$$\mathsf{wperm}(a, a_0, e) \;\wedge\; a' = \ldots \;\rightarrow\; \mathsf{wperm}(a', a_0, e)$$

implies the validity of the VC

$$\begin{aligned}(\forall e.\ (\exists i.\ a[i] = e) \;\leftrightarrow\; (\exists j.\ a_0[j] = e)) \;\wedge\; a' = \ldots \\ \rightarrow\; (\forall e.\ (\exists i.\ a'[i] = e) \;\leftrightarrow\; (\exists j.\ a_0[j] = e))\ .\end{aligned}$$

(c) Can this approximation be used to prove the weakest permutation property of

(i) InsertionSort (Figure 6.8)?

(ii) MergeSort (Figure 6.9)?

(iii) QuickSort (Section 6.2)?

If so, prove it. If not, explain why not.

6.6 (Sets with arrays). Implement an API (**application programming interface**) for manipulating sets. The underlying data structure of the implementation is arrays.

(a) Prove the correctness of the union function of Figure 6.11 by adding inductive annotations.

(b) Implement and specify and prove the correctness of an intersection function, which takes two arrays a_0 and b_0 and returns the intersection of the sets they represent.

(c) Implement and specify and prove the correctness of a subset function, which takes two arrays a_0 and b_0 and returns `true` iff the first set, represented by a_0, is a subset of the second set, represented by b_0.

6.7 (Sets with sorted arrays). Implement an API for manipulating sets. The underlying data structure of the implementation is sorted arrays.

(a) Prove the correctness of the union function of Figure 6.12 by adding inductive annotations.

(b) Implement and specify and prove the correctness of an intersection function, which takes two sorted arrays a_0 and b_0 and returns the intersection of the sets they represent as a sorted set.
(c) Implement and specify and prove the correctness of a subset function, which takes two sorted arrays a_0 and b_0 and returns `true` iff the first set, represented by a_0, is a subset of the second set, represented by b_0.

6.8 (QuickSort halts). Provide the basic paths and verification conditions for the proof of Section 6.2.2 that QuickSort always halts.

6.9 (Intuitive ranking functions). Following the proof that the recursion of qsort halts, move the location of the ranking function annotations in the following functions to produce more intuitive arguments:

(a) BinarySearch, Figure 5.2
(b) BubbleSort, Figure 5.3
(c) InsertionSort, Figure 6.8

6.10 (Fewer annotations). Notice in the annotated BubbleSort of Figure 5.17 that there are only a finite number of basic paths from function entry to L_2, from L_2 to function exit, and from L_2 back to itself.

(a) List these basic paths.
(b) Annotate only the inner loop of BubbleSort so that the VCs corresponding to the basic paths of **(a)** are valid.
(c) Treat InsertionSort of Figure 5.25 similarly.
(d) Similarly, annotate only the inner loop of BubbleSort with a ranking annotation.
(e) Treat InsertionSort similarly.

```
@pre ⊤
@post sorted(rv, 0, |rv| − 1)
int[] MergeSort(int[] a) {
  return ms(a, 0, |a| − 1);
}

@pre ⊤
@post ⊤
int[] ms(int[] a0, int ℓ, int u) {
  int[] a := a0;
  if (ℓ ≥ u) return a;
  else {
    int m := (ℓ + u) div 2;
    a := ms(a, ℓ, m);
    a := ms(a, m + 1, u);
    a := merge(a, ℓ, m, u);
    return a;
  }
}

@pre ⊤
@post ⊤
int[] merge(int[] a0, int ℓ, int m, int u) {
  int[] a := a0, buf := new int[u − ℓ + 1];
  int i := ℓ, j := m + 1;
  for @ ⊤
    (int k := 0; k < |buf|; k := k + 1) {
    if (i > m) {
      buf[k] := a[j];
      j := j + 1;
    } else if (j > u) {
      buf[k] := a[i];
      i := i + 1;
    } else if (a[i] ≤ a[j]) {
      buf[k] := a[i];
      i := i + 1;
    } else {
      buf[k] := a[j];
      j := j + 1;
    }
  }
  for @ ⊤
    (k := 0; k < |buf|; k := k + 1) {
    a[ℓ + k] := buf[k];
  }
  return a;
}
```

Fig. 6.9. MergeSort

```
define int e = ?;

@pre ⊤
@post wperm(a, a_0, e)
int[] BubbleSort(int[] a_0) {
  int[] a := a_0;
  for
    @L_1 : −1 ≤ i < |a| ∧ wperm(a, a_0, e)
    (int i := |a| − 1; i > 0; i := i − 1) {
    for
      @L_2 : 0 ≤ j < i ∧ i < |a| ∧ wperm(a, a_0, e)
      (int j := 0; j < i; j := j + 1) {
      if (a[j] > a[j + 1]) {
        int t := a[j];
        a[j] := a[j + 1];
        a[j + 1] := t;
      }
    }
  }
  return a;
}
```

Fig. 6.10. BubbleSort with annotations for weak permutation

```
define int e = ?;

@pre ⊤
@post [(∃i. 0 ≤ i < |rv| ∧ rv[i] = e)
       ↔ (∃i. 0 ≤ i < |a_0| ∧ a_0[i] = e) ∨ (∃i. 0 ≤ i < |b_0| ∧ b_0[i] = e)]
int[] union(int[] a_0, int[] b_0) {
  int[] u := new int[|a_0| + |b_0|];
  int j := 0;
  for @ ⊤
    (int i = 0; i < |a_0|; i := i + 1) {
    u[j] := a_0[i];
    j := j + 1;
  }
  for @ ⊤
    (int i = 0; i < |b_0|; i := i + 1) {
    u[j] := b_0[i];
    j := j + 1;
  }
  return u;
}
```

Fig. 6.11. Function union of the linear set implementation

```
define int e = ?;

@pre sorted(a_0, 0, |a_0| − 1) ∧ sorted(b_0, 0, |b_0| − 1)
@post [ sorted(rv, 0, |rv| − 1)
      ∧ [ (∃i. 0 ≤ i < |a_0| ∧ a_0[i] = e) ∨ (∃i. 0 ≤ i < |b_0| ∧ b_0[i] = e)
        ↔ (∃i. 0 ≤ i < |rv| ∧ rv[i] = e) ] ]
int[] union(int[] a_0, int[] b_0) {
  int[] u := new int[|a_0| + |b_0|];
  int i := 0, j := 0;
  for @ ⊤
    (int k = 0; k < |u|; k := k + 1) {
    if (i ≥ |a_0|) {
      u[k] := b_0[j];
      j := j + 1;
    }
    else if (j ≥ |b_0|) {
      u[k] := a_0[i];
      i := i + 1;
    }
    else if (a_0[i] ≤ b_0[j]) {
      u[k] := a_0[i];
      i := i + 1;
    }
    else {
      u[k] := b_0[j];
      j := j + 1;
    }
  }
  return u;
}
```

Fig. 6.12. Function union of the sorted set implementation

Part II

Algorithmic Reasoning

It is reasonable to hope that the relationship between computation and mathematical logic will be as fruitful in the next century as that between analysis and physics in the last. The development of this relationship demands a concern for both applications and mathematical elegance.

— John McCarthy
A Basis for a Mathematical Theory of Computation, 1963

Having established in Part I the mathematical foundations for precision in designing and implementing software and hardware systems, we turn in Part II to the task of automating some of the required logical reasoning.

Chapters 7–10 discuss decision procedures for many of the theories and fragments of theories introduced in Chapter 3. These decision procedures automate the task of proving the verification conditions of Chapters 5 and 6. Chapter 7 applies the method of quantifier elimination to decide validity in $T_{\mathbb{Z}}$ and $T_{\mathbb{Q}}$. Chapters 8 and 9 focus on decision procedures for the quantifier-free fragments of the theories $T_{\mathbb{Q}}$, T_{E}, T_{cons}, and T_{A}. In Chapter 10, these procedures are combined in the Nelson-Oppen framework to address formulae with symbols of multiple signatures. The treatment of arrays in Chapter 11 looks beyond the quantifier-free fragment, but we show how the combination method of Chapter 10 still applies.

Chapter 12 turns to automating the inductive assertion method of Chapter 5. It presents a methodology for constructing algorithms that learn inductive facts about software. These algorithms reduce the burden on the engineer to provide annotations.